

TAM 470 Project 1 Report

Hongbo Nie

2024.10.30

Project problem description:

Double spring pendulum problem description

The double spring pendulum in Figure 1 consists of two particles of mass m_1 and m_2 connected by linear springs. The entire system is mounted at a frictionless pin support at the coordinate system origin $(0, 0)$. The force in each spring is given by $F = -k_i(L_i - L_{0i})$, where k_i is the spring constant, L_{0i} is the equilibrium spring length, and L_i is the current length ($i = 1, 2$ for spring 1 and 2 labeled in the figure). The direction of spring force on each particle is along the orientation of the connected spring(s), and the force is compressive when $L_i < L_{0i}$ and tensile if $L_i > L_{0i}$. The total force on each particle is equal to the vector sum of the forces from the attached springs, plus the gravitational force of $m_i g$ in the negative y -direction. In analyzing this system, you can ignore the weight of the springs and any air resistance on the springs on particles. Additionally, you can assume that the end of each spring has a pinned connection to each mass that allows rotation without any friction.

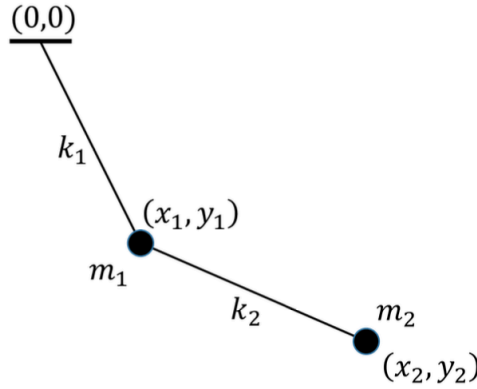


Figure 1: Diagram of the double-spring pendulum system.

Question 1:

Draw a free-body diagram for each mass in the double spring pendulum and use it to derive the equations of motion for the system in the form $\mathbf{u}' = \mathbf{f}(\mathbf{u})$ where $\mathbf{u} = (x_1, y_1, x_2, y_2, x_1', y_1', x_2', y_2')$.

Question 1 Solution:

Free body diagrams of each mass:

From the coordinate diagram, since positive y direction is point upward, therefore, y_1 and y_2 are in the negative direction and should add minus sign in front of them.

Spring 1 force: $T_1 = k_1 (L_1 - L_{01})$

$L_1 = \sqrt{x_1^2 + y_1^2}$

x -direction: $T_{1x} = (k_1 (L_1 - L_{01})) \cdot \frac{x_1}{L_1}$

y -direction: $T_{1y} = (k_1 (L_1 - L_{01})) \cdot (-\frac{y_1}{L_1})$

Spring 2 force: $T_2 = k_2 (L_2 - L_{02})$

$L_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

x -direction: $T_{2x} = k_2 (L_2 - L_{02}) \cdot \frac{(x_2 - x_1)}{L_2}$

y -direction: $T_{2y} = k_2 (L_2 - L_{02}) \cdot (-\frac{(y_2 - y_1)}{L_2})$

From free body diagram of each mass, we can derive the equations of motion:

For m_1 :

x -direction: $-T_{1x} + T_{2x} = m_1 \ddot{x}_1$ ($F = ma$)

$-k_1 (L_1 - L_{01}) \cdot \frac{x_1}{L_1} + k_2 (L_2 - L_{02}) \cdot \frac{(x_2 - x_1)}{L_2} = m_1 \ddot{x}_1$

$\Rightarrow \ddot{x}_1 = \frac{-k_1 (L_1 - L_{01}) \cdot \frac{x_1}{L_1} + k_2 (L_2 - L_{02}) \cdot \frac{(x_2 - x_1)}{L_2}}{m_1}$

P₁

Continue:

y-direction:

$$\cancel{T_1y} - T_2y - m_1g = m_1\ddot{y}_1$$

$$k_1(L_1 - L_{01}) \cdot \left(-\frac{y}{L_1}\right) - k_2(L_2 - L_{02}) \cdot \left(-\frac{(y_2 - y_1)}{L_2}\right) - m_1g = m_1\ddot{y}_1$$

$$\Rightarrow \ddot{y}_1 = \frac{k_1(L_1 - L_{01}) \cdot \left(-\frac{y}{L_1}\right) - k_2(L_2 - L_{02}) \cdot \left(-\frac{(y_2 - y_1)}{L_2}\right) - m_1g}{m_1}$$

For m_2 :

x-direction:

$$-T_2x = m_2\ddot{x}_2$$

$$-k_2(L_2 - L_{02}) \cdot \frac{(x_2 - x_1)}{L_2} = m_2\ddot{x}_2$$

$$\Rightarrow \ddot{x}_2 = \frac{-k_2(L_2 - L_{02}) \cdot \frac{(x_2 - x_1)}{L_2}}{m_2}$$

y-direction:

$$T_2y - m_2g = m_2\ddot{y}_2$$

$$k_2(L_2 - L_{02}) \cdot \left(-\frac{(y_2 - y_1)}{L_2}\right) - m_2g = m_2\ddot{y}_2$$

$$\Rightarrow \ddot{y}_2 = \frac{k_2(L_2 - L_{02}) \cdot \left(-\frac{(y_2 - y_1)}{L_2}\right) - m_2g}{m_2}$$

$$u_1 \quad u_2 \quad u_3 \quad u_4 \quad u_5 \quad u_6 \quad u_7 \quad u_8$$

$$U = [x_1, y_1, x_2, y_2, x_1', y_1', x_2', y_2']$$

$$u_1' = x_1' = u_5, \quad u_2' = y_1' = u_6, \quad u_3' = x_2' = u_7, \quad u_4' = y_2' = u_8$$

continue:

$$u_5' = x_1'' = \frac{-k_1(L_1 - l_{01}) \cdot \frac{x_1}{L_1} + k_2(L_2 - l_{01}) \cdot \frac{(x_2 - x_1)}{L_2}}{m_1}$$

$$u_6' = y_1'' = \frac{k_1(L_1 - l_{01}) \cdot \left(\frac{-y_1}{L_1}\right) - k_2(L_2 - l_{01}) \cdot \left(\frac{-(y_2 - y_1)}{L_2}\right) - m_1 g}{m_1}$$

$$u_7' = x_2'' = \frac{-k_2(L_2 - l_{02}) \cdot \frac{(x_2 - x_1)}{L_2}}{m_2}$$

$$u_8' = y_2'' = \frac{k_2(L_2 - l_{02}) \cdot \left(\frac{-(y_2 - y_1)}{L_2}\right) - m_2 g}{m_2}$$

$$u' = \begin{bmatrix} u_5, u_6, u_7, u_8, x_1'', y_1'', x_2'', y_2'' \end{bmatrix} = f(u)$$

$$= f(x_1, y_1, x_2, y_2, x_1', y_1', x_2', y_2')$$

Question 2:

To answer this question, you will need to complete the following functions using the code template hosted on PrairieLearn (descriptions of required inputs and outputs are on PrairieLearn):

1. `fpend` to compute the right-hand side ODE function in $\mathbf{u}' = \mathbf{f}(\mathbf{u})$
2. `rk4pend` to implement the RK4 time integrator
3. `ab2pend` to implement the AB2 time integrator
4. `lfpnd` to implement the Leapfrog time integrator
5. `pendulumEnergy` to compute the total (kinetic + potential) energy of the system.

Consider the following inputs for the system initial conditions and properties:

```
import numpy as np

u0 = np.array([1, 0, 1, 1, 0, 0, 0, 0])
ks = np.array([500, 500])
L0 = np.array([1.0, 1.0])
m = np.array([1.0, 1.0])
g = 9.81
```

Answer the following questions based on these inputs:

1. (20 pts) For a simulation time range of $t \in [0, 600]$, create plots of total system energy vs time for the RK4, AB2, and Leapfrog schemes, each with three different simulation time step sizes of $h = 0.02$, $h = 0.01$, and $h = 0.005$. Choose appropriate vertical axis limits that make it easy to compare the performance of the methods to one another, and be sure to clearly label/title each plotted result with the corresponding scheme and time step used. **This is a total of 9 separate plots.**
2. (5 pts) Based on your plots, which schemes appear to be stable for the time step sizes used? Comment on whether the stability results seem reasonable based on what you know regarding the stability properties of these 3 schemes.
3. (10 pts) Leapfrog is less accurate than RK4 but is known for favorable time-averaged energy conservation properties in conservative systems like this one. Smooth out the Leapfrog energy response by time-averaging the system energy over intervals in time, and create plots that compare these time-averaged Leapfrog energy results to the RK4 (unaveraged) energy results. Comment on any trends you notice in energy conservation vs time for Leapfrog vs RK4, including any dependence on time step size h .

Question 2 Solution:

Part 1:

The plots of total energy vs time for RK4, AB2, and Leapfrog schemes are shown in Figure 1, 2, 3. With each figure shows 3 plots of total energy vs time for RK4, AB2, and Leapfrog schemes under specific time step size. Figure 1 shows the plots with time step size of $h=0.02$, Figure 2 shows the plots with time step size of $h=0.01$, Figure 3 shows the plots with time step size of $h=0.005$.

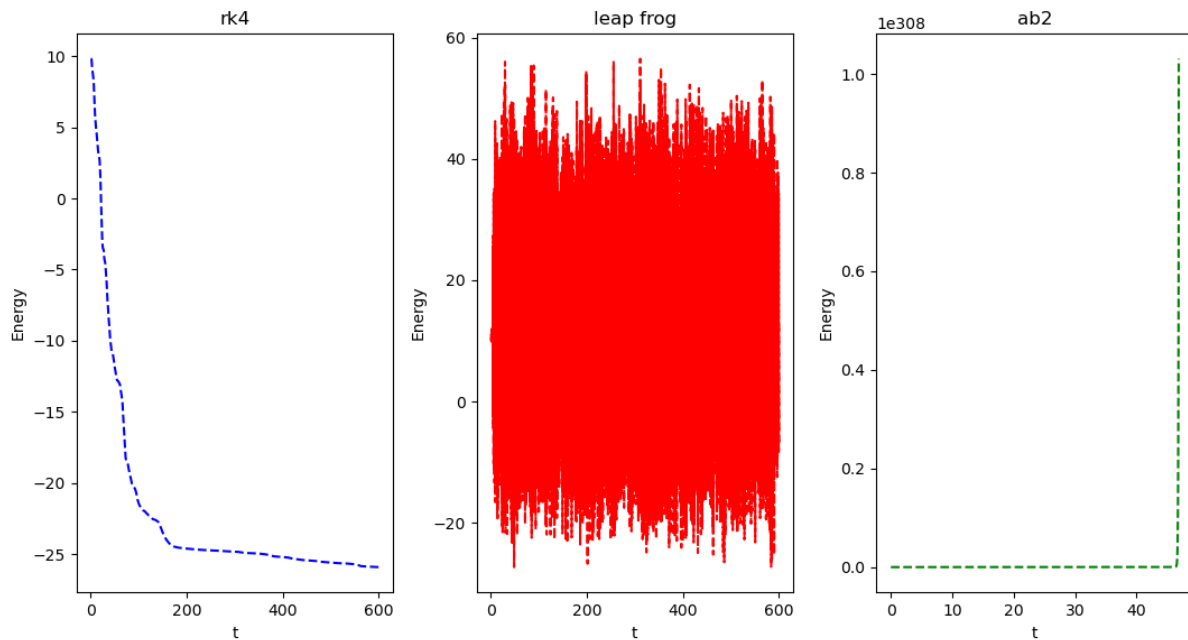


Figure 1. Total Energy vs. Time with $h=0.02$

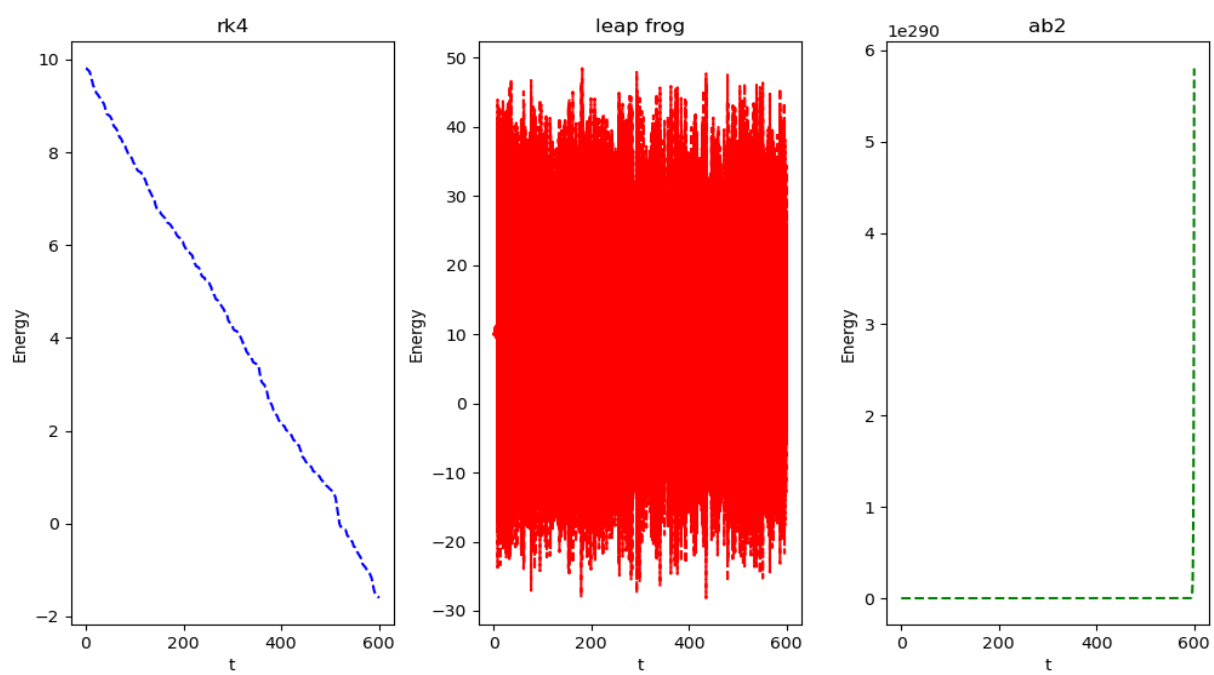


Figure 2. Total Energy vs. Time with $h=0.01$

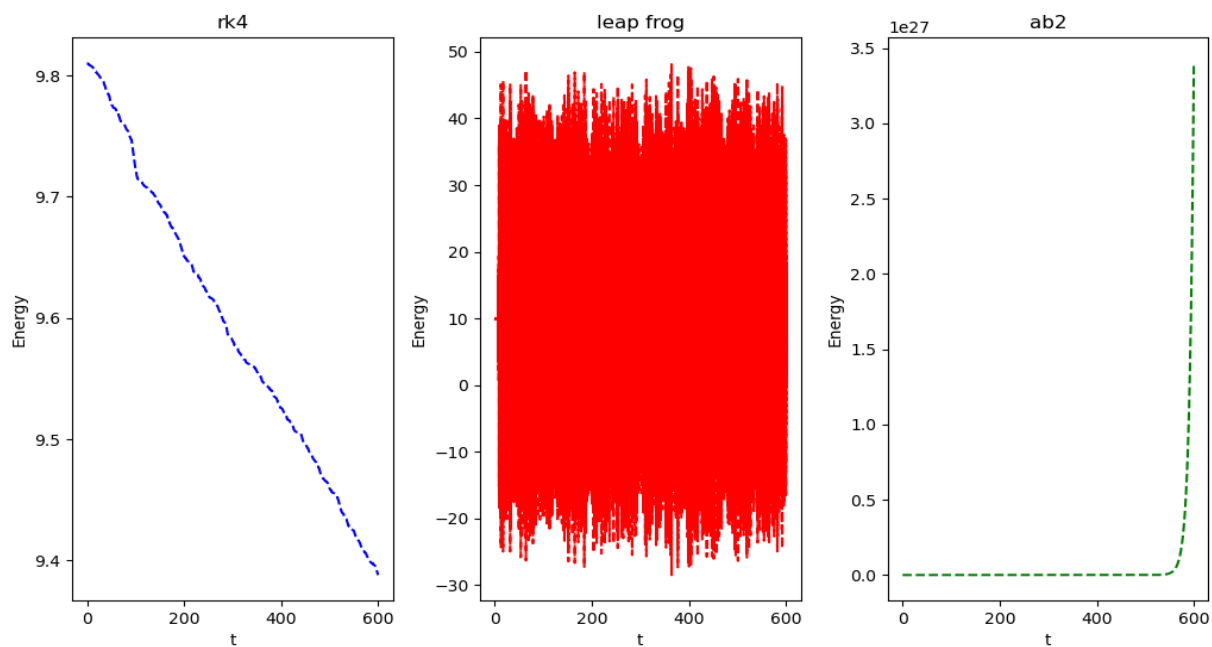


Figure 3. Total Energy vs. Time with $h=0.005$

Part 2:

For Figure 1 with time step size of $h=0.02$.

The left most RK4 plot shows the energy drops quickly from 10J to -25J, and stabilizes after $t=200$, which suggests this scheme is relatively stable with time step size of $h=0.02$. However, the significant drop in energy suggests that the RK4 is not an energy conservative scheme.

The middle Leapfrog plot shows the energy oscillates between 60J and -30J and doesn't blow up which suggests Leapfrog is relatively stable.

The right most AB2 plot shows the energy blows up to $1e308J$ at $t=600$, which indicates the energy value blows up over time, and the AB2 is unstable.

For Figure 2 with time step size of $h=0.01$.

The left most RK4 plot shows the total energy drops continuously from 10J to -2J over time range (0,600), in such big time range, the energy change is not very extreme which suggests this scheme is relatively stable with time step size of $h=0.01$. However, the drop in energy suggests that the RK4 is not an energy conservative scheme.

The middle Leapfrog plot shows the total energy oscillates between 50J and -30J and doesn't blow up which suggests Leapfrog is relatively stable.

The right most AB2 plot shows the energy blows up to $1e290J$ at $t=600$, which indicates the energy value blows up over time, and the AB2 is unstable.

For Figure 3 with time step size of $h=0.005$

The left most RK4 plot shows the energy stays relatively stable in the range of 9.8J to 9.4J, which suggests this scheme is relatively stable with time step size of $h=0.005$.

The middle Leapfrog plot shows the total energy oscillates between 50J and -30J and doesn't blow up which suggests Leapfrog is relatively stable.

The right most AB2 plot shows the energy blows up to $1e27J$ at $t=600$, which indicates the energy value blows up over time, and the AB2 is unstable.

In conclusion, for $h=0.02$, RK4 and Leapfrog are stable, AB2 is unstable, for $h=0.01$, RK4 and Leapfrog are stable, AB2 is unstable, for $h=0.005$, RK4 and Leapfrog are stable, AB2 is unstable.

RK4 is stable for every condition is reasonable since the RK4 method is generally stable for a relatively wide range of time step sizes h , it is stable as long as $h \leq \frac{2.79}{|\lambda|}$. Leapfrog is stable when h is relatively small, since the time step $h=0.02$, $h=0.01$, $h=0.005$ are relatively small, so Leapfrog is stable is reasonable. For AB2, since it is an explicit two-step method, its stability limit is often narrower, so it is unstable is reasonable.

Part 3:

To examine the average total energy, we divide the leapfrog total energy array into intervals with size 10, calculate average values of each interval (average energy of each interval) and store the values into a new array. Then plot the graph of this new array vs. time will give us plot of the average total energy of the leapfrog vs. time. Figure 4 shows the plot that compare the time-averaged Leapfrog energy results to the RK4 energy results.

The average total energy of each time step size are as follows:

Leapfrog average total energy at $h=0.02$ is 10.318036534510213J

Leapfrog average total energy at $h=0.01$ is 10.318036534510213J

Leapfrog average total energy at $h=0.005$ is 9.931774499388125J

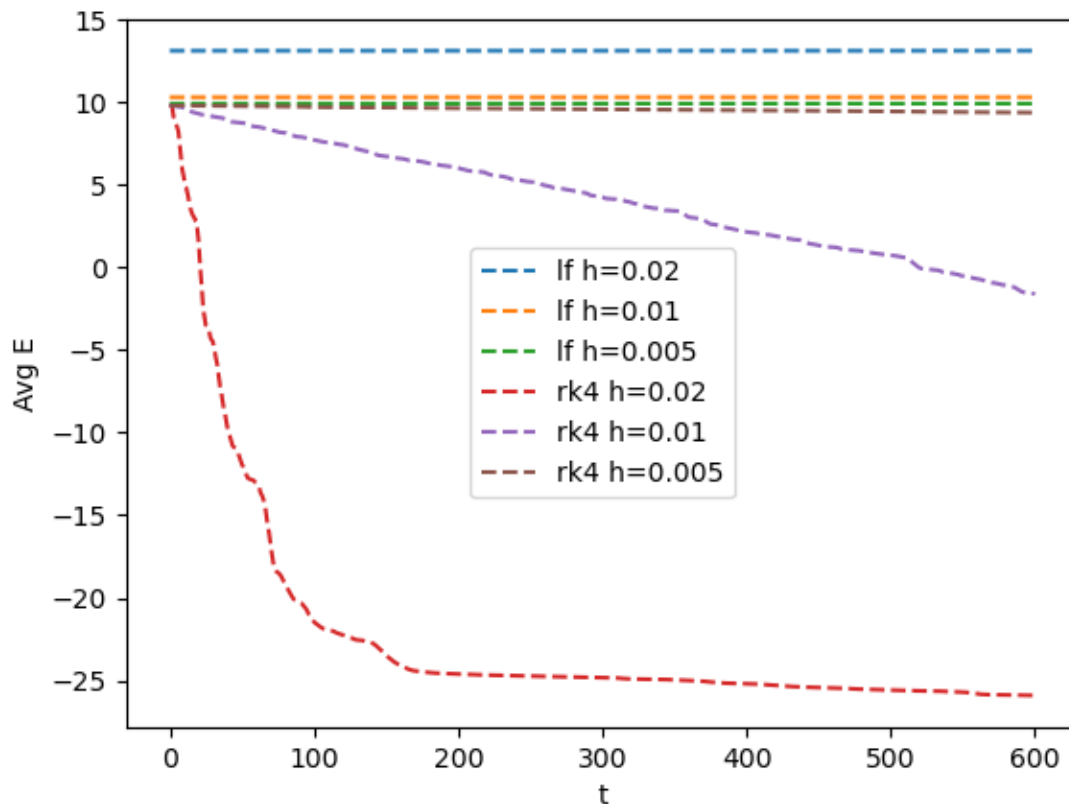


Figure 4. Average total energy vs time at $h=0.02$

From Figure 4, it is obvious that time-averaged Leapfrog energy is stable for all different time step. RK4 energy is stable for $h=0.005$, RK4 energy decreases close linearly for $h=0.01$, and RK4 energy decreases quickly between $t=0$ and $t=200$, and stabilizes after $t=200$. This suggests RK4 works best with small time step size.

Question 3:

Now consider replacing the springs with stiffer ones, corresponding to the simulation settings below:

```
import numpy as np

u0 = np.array([1, 0, 1, 1, 0, 0, 0, 0])
ks = np.array([1e6, 1e6])
L0 = np.array([1.0, 1.0])
m = np.array([1.0, 1.0])
g = 9.81
```

This will introduce a large difference in the two natural time scales of the system: the time scale that depends on the frequency of vibrations in the springs (this is higher for stiffer springs), and the time scale related to the frequency of pendulum swinging motion (depends on masses and spring equilibrium lengths).

For this stiff system, we will explore the performance of `scipy.solve_ivp` to solve the ODE using the RK45 (default choice, explicit) and Radau (implicit, stiff) solvers. You will find a code template on PrairieLearn to assist with the syntax needed to invoke these solvers for this problem.

1. (6 pts) Use the RK45 and Radau solvers to generate a plots of total system energy vs time. You may wish to create two separate plots to clearly display the behavior of each method.
2. (9 pts) Comment on the performance of the methods by considering: do they both produce reasonable results when looking at the animated motion of the system? Do they both produce good results considering total energy conservation? What time step size (e.g., average or typical value over the course of the simulation) is used by each method? Which method appears to be the best choice to use for this system?

Question 3 Solution:

Part 1:

Figure 5, 6 shows the plot of total system energy vs time using `scipy.solve_ivp` using RK45 and Radau solvers respectively.

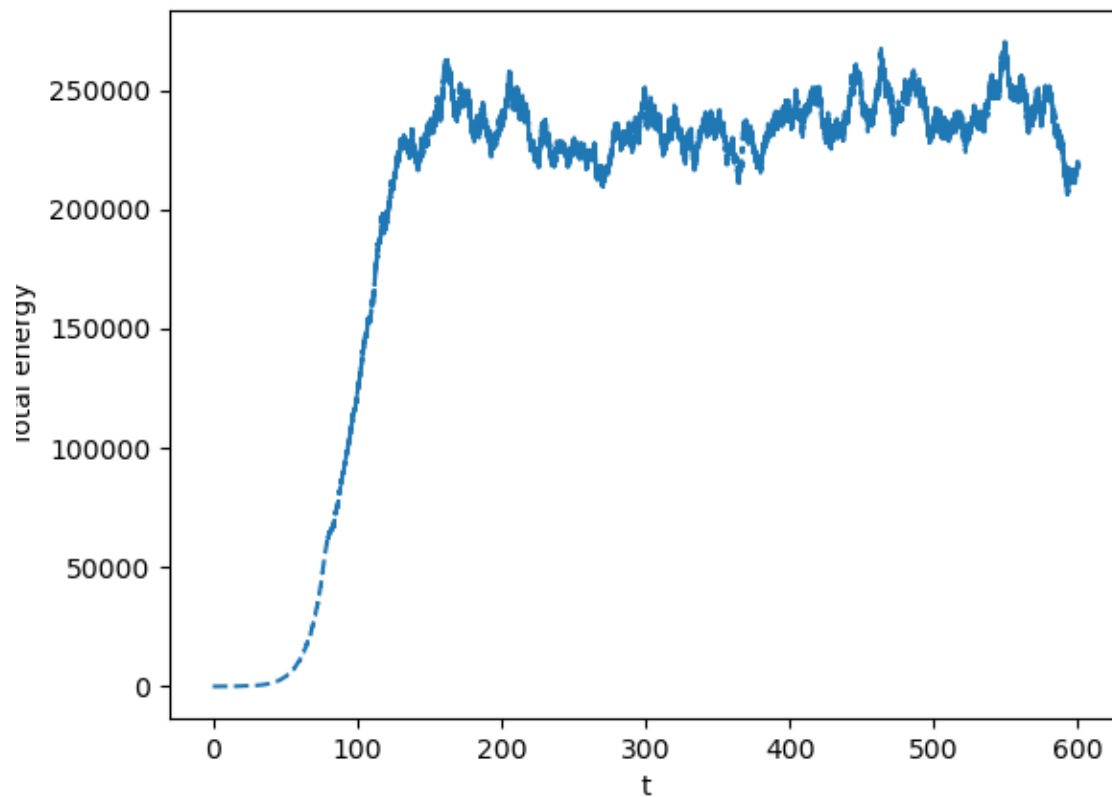


Figure 5. Total system energy vs time with RK45 solver

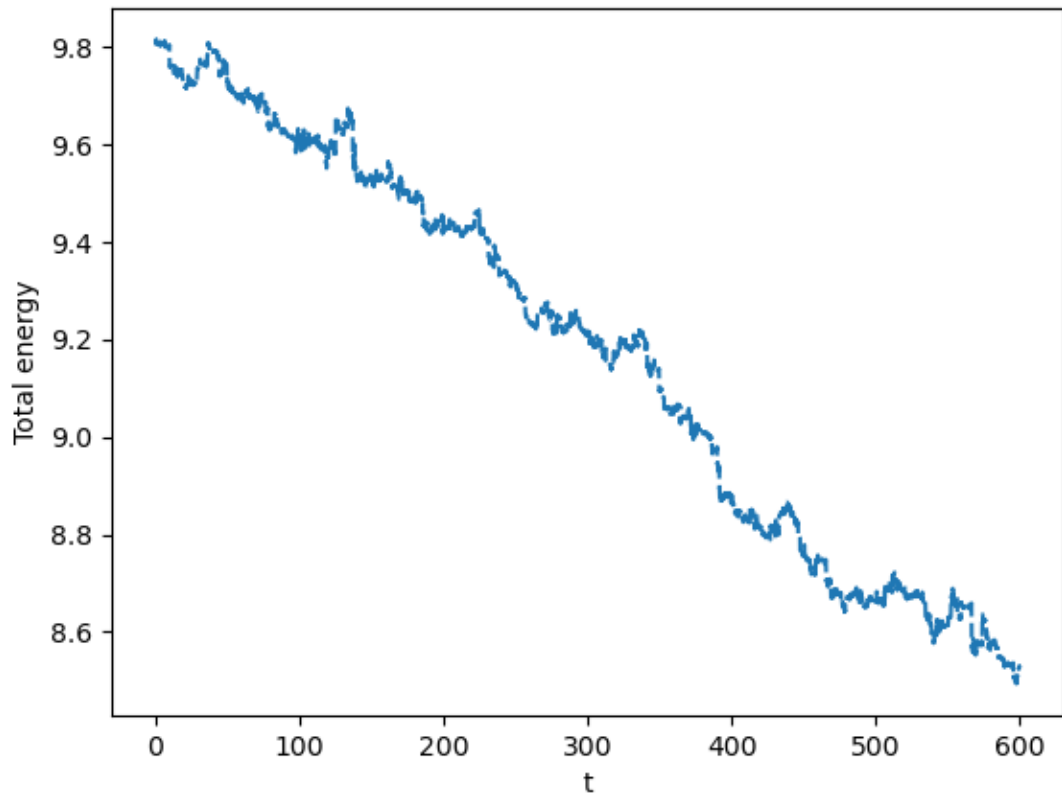


Figure 6. Total system energy vs time with Radau solver

Part 2:

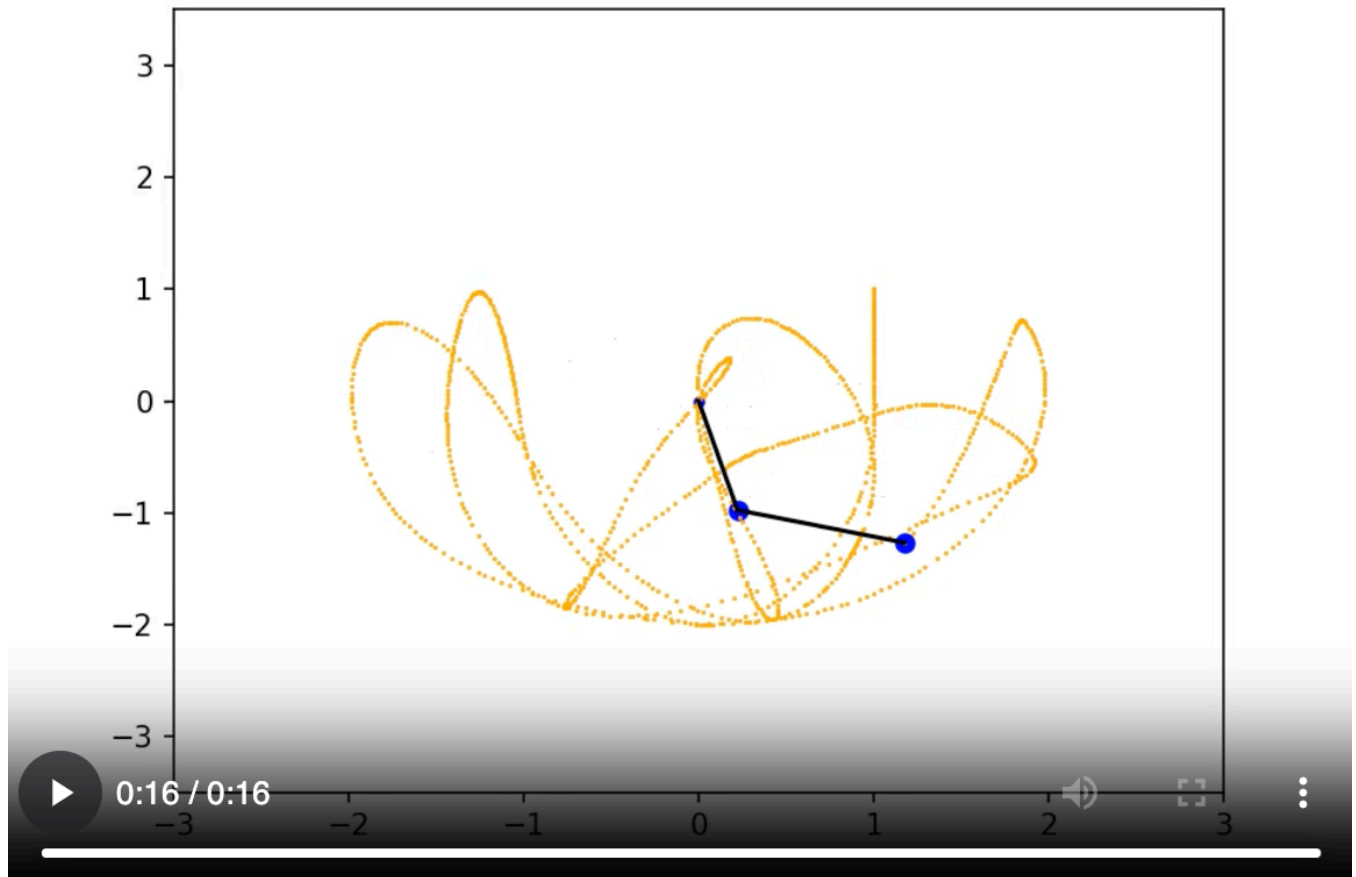


Figure 7. Animation of Radau method

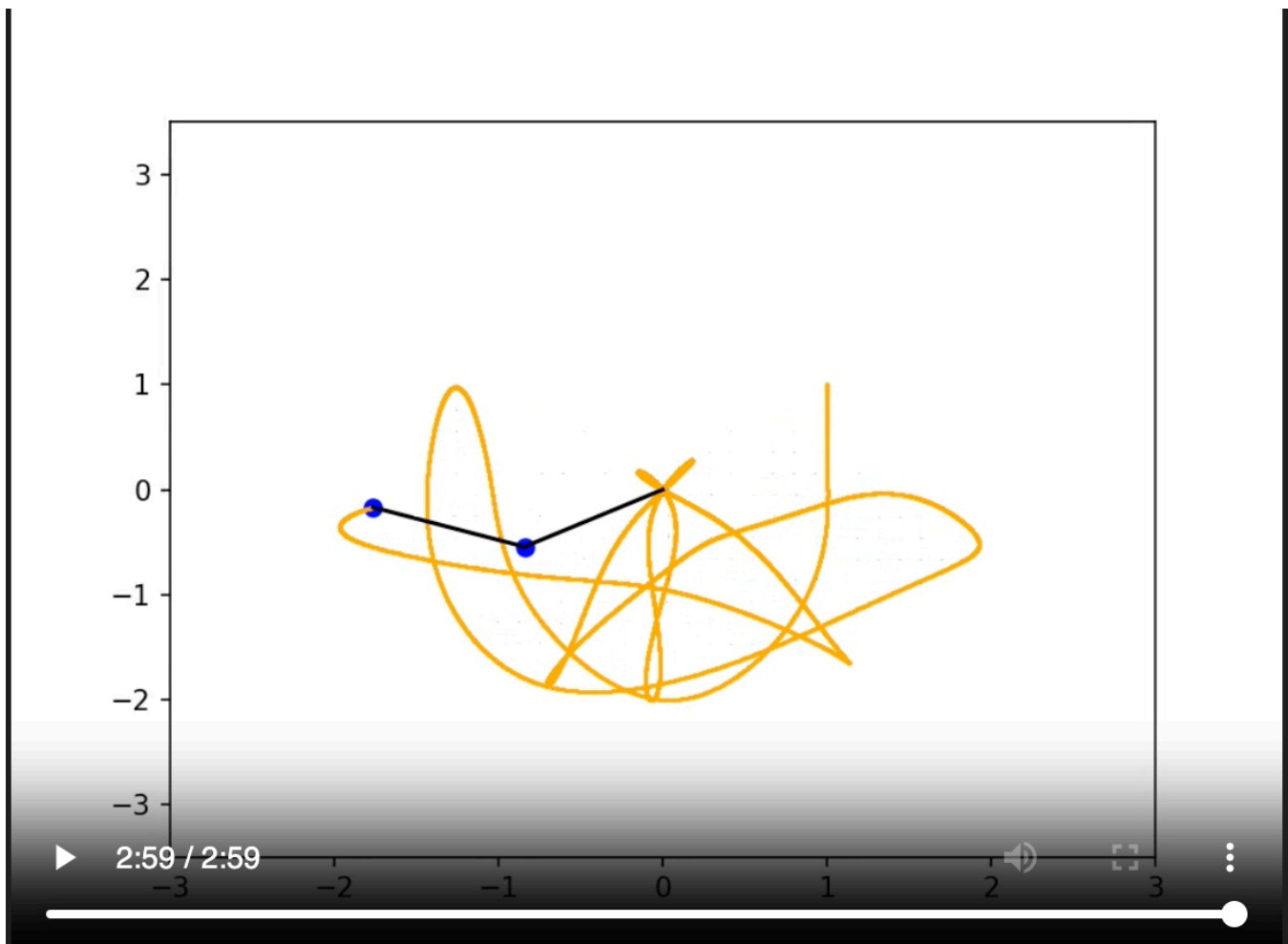


Figure 8. Animation of RK45 Method

From the animation, both Radau and Rk45 shows reasonable result.

RK45 method create a total energy curve that increase significantly from around 10J to around 250000J initially and then stabilizes over time. The energy increment is too big that can conclude that the energy solution explode and lead to unstable, eventhough it becomes stabilized later.

Radau method create a total energy curve that decrease continuesly from around 9.8J to 8.5J over time interval (0,600), which is stable, and states the method follows energy conservation.

RK45 can optimize the time step for accuracy and stability but require small time step for stiff problems. On the other hand, Radau is an adaptive implicit

solver which can take larger time steps and is suitable for stiff problems. In the case of the project problem, Radau method is the best choice to use.